

Module 2:Transport Layer and Network Layer



Module 2: Transport Layer and Network Layer

Transport Layer

- Services and Protocols
- UDP and TCP

Hands-on

- Sockets Introduction
- Elementary TCP Sockets
- TCP Client/Server Example
- I/O Multiplexing
 - select()
 - poll()
- Elementary UDP Sockets
- Advanced I/O Functions

Network Layer

- Introduction
- Network-layer Protocols
- Unicast Routing
- Multicast Routing
- Multicasting Basics
- Intra-domain Routing
- Inter-domain Routing
- Next Generation IP (IPv6)
- Quality of Service (QoS)

Hands-on

- Linux Routing Table
- Routing Caches
- Routing Cache Implementation
- Adding Routes using ip

Unicast Routing

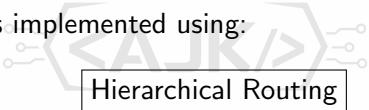
- Network layer delivers datagrams to destinations.
- Based on number of destinations:
 - Unicast Routing (One-to-One)
 - Multicast Routing (One-to-Many)
 - Broadcast Routing (One-to-All)

Focus

Unicast Routing: Delivery of a packet from one source to one destination.

Routing in the Internet

- Internet contains:
 - Huge number of hosts
 - Large number of routers
- Unicast routing is implemented using:



Hierarchical Routing

- Routing is performed in multiple stages using different routing algorithms.

General Idea of Unicast Routing

- Packets are routed hop-by-hop.
- Routers use forwarding tables.

Source Host

- Sends packet to default router.
- Does not need forwarding table.

Destination Host

- Receives packet from default router.
- Does not need forwarding table.

Routers

- Maintain forwarding tables.
- Perform routing decisions.

Source Router and Destination Router

- Source Router:
 - Default router of source host.
- Destination Router:
 - Router connected to destination network.
- Main routing problem:
 - Determine intermediate routers to visit.
 - Select the best route.

Internet as a Graph

- Internet can be modeled as a graph.
- Graph consists of:
 - Nodes
 - Edges

Routing Representation:



- Router $\rightarrow$ Node
- Network between routers $\rightarrow$ Edge

Purpose

Graph model helps determine optimal routes.

Weighted Graph Model

- Internet is represented as a weighted graph.
- Each edge has an associated cost.

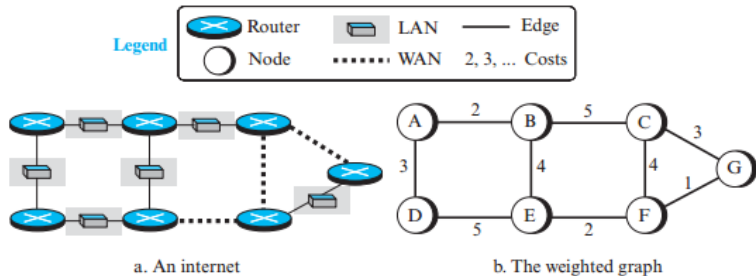
Examples of Cost:

- Distance
- Delay
- Bandwidth usage
- Administrative metric

- No edge between nodes:

$$Cost = \infty$$

Figure 4.56 *An internet and its graphical representation*



Least-Cost Routing

- Best route is interpreted as:

Least-Cost Route

- Total route cost must be minimum among all possible routes.

Example from Figure 4.56:

$$A \rightarrow B \rightarrow E$$

$$\text{Cost} = 6$$

This is the least-cost route from A to E.

Need for Least-Cost Paths

- Assume:

N = Number of routers

- Each router needs:

$$(N - 1)$$

least-cost routes.

- Entire Internet requires:


$$N(N - 1)$$

least-cost paths.

Example:

$$N = 10$$

$$10(10 - 1) = 90$$

least-cost paths.

Least-Cost Tree

- More efficient than storing all paths individually.
- Combines shortest paths into a single tree.

Definition:

Least-Cost Tree

A tree rooted at a source router that spans all routers and provides the shortest path from the root to every other node.

Properties of Least-Cost Trees

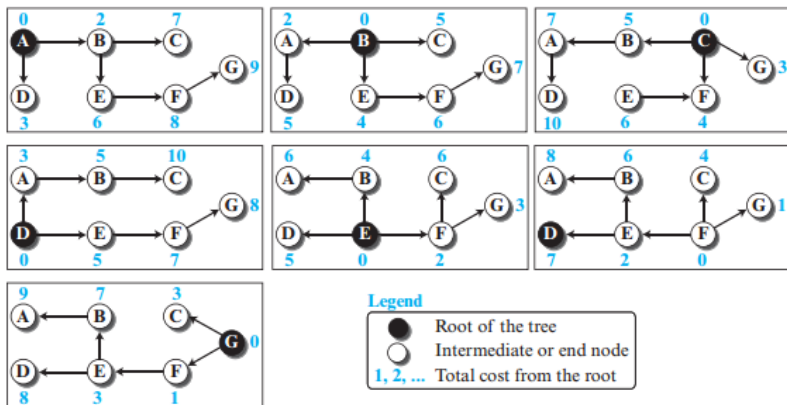
- One least-cost tree per router.
- Entire network requires:

least-cost trees.

- Tree spans all routers.
- Every path from root is a shortest path.



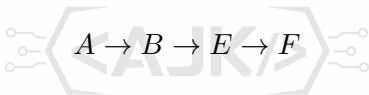
Figure 4.57 *Least-cost trees for nodes in the internet of Figure 4.56*



Property 1: Route Reversibility

- Least-cost route from X to Y is the reverse of the least-cost route from Y to X.
- Total cost remains the same.

Example:



Reverse route:

$$F \rightarrow E \rightarrow B \rightarrow A$$

Cost:

8

in both directions.

Property 2: Path Consistency

- Routing can be divided into segments.

- Travel from:

$$X \rightarrow Y$$

using X's tree.

- Then continue:

$$Y \rightarrow Z$$

using Y's tree.

- Combined route remains least-cost.



Example of Path Consistency

Direct route in A's tree:

$$A \rightarrow B \rightarrow E \rightarrow F \rightarrow G$$

Cost:

9

Segmented route:



$$A \rightarrow B \rightarrow E$$

then

$$E \rightarrow F \rightarrow G$$

Total Cost:

$$6 + 3 = 9$$

Unicast Routing: Summary

- Unicast routing provides one-to-one delivery.
- Routing decisions are made by routers.
- Internet can be modeled as a weighted graph.
- Best route is the least-cost route.
- Least-cost trees efficiently represent shortest paths.

Unicast Routing = Forwarding Tables + Least-Cost Paths

Routing Algorithms

- Routing algorithms are used to build least-cost paths.
- Different algorithms differ in:
 - Interpretation of cost
 - Method of building least-cost trees
- One common algorithm:

Distance Vector (DV) Routing

Distance Vector Routing

- Each router creates its own least-cost tree.
- Initially uses information from immediate neighbors only.
- Routers exchange routing information with neighbors.
- Trees gradually become complete.

Idea

A router continuously tells its neighbors what it knows about the network.

Bellman-Ford Equation

Distance Vector Routing is based on the Bellman-Ford Equation.

$$D_{xy} = \min\{(c_{xa} + D_{ay}), (c_{xb} + D_{by}), (c_{xc} + D_{cy}), \dots\}$$

where

- D_{xy} = shortest distance from X to Y
- c_{xy} = direct cost between nodes

Purpose: Find the least-cost path through intermediate nodes.

Simplified Bellman-Ford Equation

For updating routes through a neighbor z :

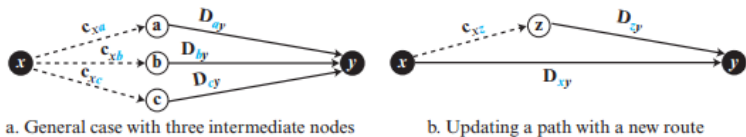
$$D_{xy} = \min\{D_{xy}, (c_{xz} + D_{zy})\}$$

- Compare existing route cost.
- Compare route through neighbor.
- Keep the smaller value.

Table 4.4 *Distance-Vector Routing Algorithm for A Node*

```
1 Distance_Vector_Routing ( )
2 {
3     // Initialize (create initial vectors for the node)
4     D[myself] = 0
5     for (y = 1 to N)
6     {
7         if (y is a neighbor)
8             D[y] = c[myself][y]
9         else
10            D[y] = ∞
11    }
12    send vector {D[1], D[2], ..., D[N]} to all neighbors
13    // Update (improve the vector with the vector received from a neighbor)
14    repeat (forever)
15    {
16        wait (for a vector  $D_w$  from a neighbor w or any change in the link)
17        for (y = 1 to N)
18        {
19            D[y] = min [D[y], (c[myself][w] +  $D_w[y]$ )] // Bellman-Ford equation
20        }
21        if (any change in the vector)
22            send vector {D[1], D[2], ..., D[N]} to all neighbors
23    }
24 } // End of Distance Vector
```

Figure 4.58 Graphical idea behind Bellman-Ford equation



Distance Vector

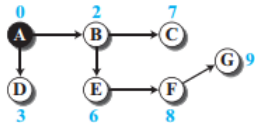
- A least-cost tree contains shortest paths.
- Distance Vector stores only the costs.

Example:


$$DV(A) = [0 \quad 2 \quad 5 \quad 3 \quad 6 \quad 8 \quad 9]$$

- Index = Destination
- Value = Least cost

Figure 4.59 *The distance vector corresponding to a tree*



a. Tree for node A

A	
A	0
B	2
C	7
D	3
E	6
F	8
G	9

b. Distance vector for node A

Creating Initial Distance Vectors

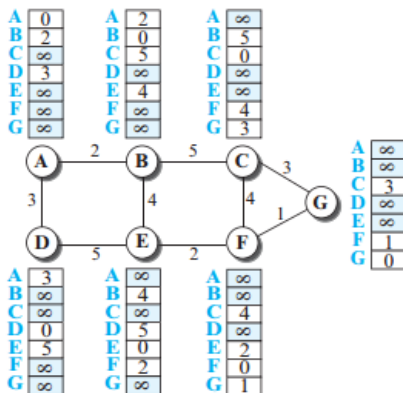
Each router:

- 1 Discovers immediate neighbors.
- 2 Measures cost to each neighbor.
- 3 Creates a rudimentary vector.

Known Neighbor Cost $\rightarrow$ Actual Value

Unknown Node $\rightarrow \infty$

Figure 4.60 *The first distance vector for an internet*



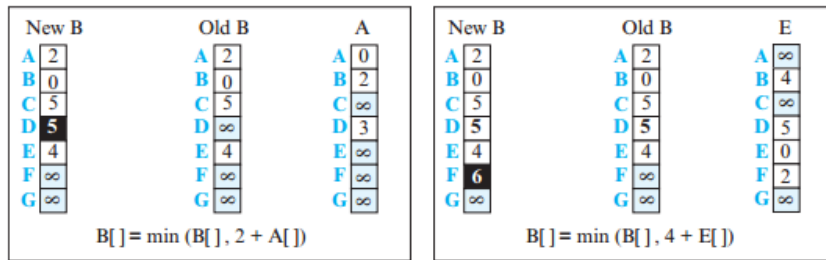
Updating Distance Vectors

- Routers exchange vectors with neighbors.
- Bellman-Ford equation updates costs.
- Improved vectors are immediately shared again.

Benefits:

- Gradually discovers shortest paths.
- Eventually reaches stable routing information.

Figure 4.61 *Updating distance vectors*



a. First event: B receives a copy of A's vector.

b. Second event: B receives a copy of E's vector.

Note:

X[]: the whole vector

Count-to-Infinity Problem

- Good news travels fast.
- Bad news travels slowly.

If a link fails:

- Routers continue advertising outdated paths.
- Costs increase gradually.
- Convergence becomes slow.

$$1 \rightarrow 2 \rightarrow 3 \rightarrow \dots \rightarrow \infty$$

This problem is called:

Count to Infinity

Two-Node Loop Problem

- Link between A and X fails.
- B still believes A can reach X.
- A then believes B can reach X.

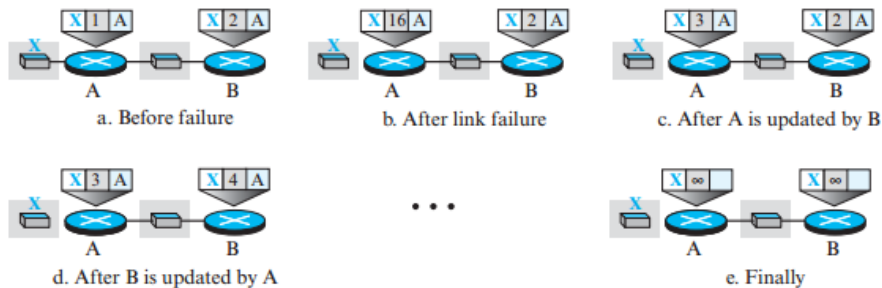
Result:



Packets keep bouncing between routers.

Routing Loop

Figure 4.62 *Two-node instability*



Solutions to Routing Loops

Split Horizon

- Do not advertise a route back to the router from which it was learned.

Poisoned Reverse

- Advertise the route with cost = ∞ .
- Warns neighbor not to use that route.

Limitation

- Two-node loops can be solved.
- Three-node instability may still occur.

Distance Vector Routing: Summary

- Uses Bellman-Ford Equation.
- Routers exchange routing information with neighbors.
- Routing information stored as Distance Vectors.
- Gradually builds least-cost paths.
- Suffers from Count-to-Infinity problem.
- Split Horizon and Poisoned Reverse reduce instability.

DV Routing = Neighbor Exchange + Bellman-Ford Updates

Link-State Routing (LS)

- Routing algorithm based on least-cost routing.
- Uses the state of each link (edge) in the network.
- Link state is represented by its cost.
- Lower cost links are preferred.
- Infinite cost indicates a broken or non-existent link.

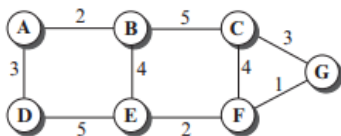
Key Idea

Every router builds a complete map of the network and computes its own least-cost tree.

Link-State Database (LSDB)

- LSDB = Collection of states of all links.
- Represents the complete network topology.
- Same LSDB is maintained by every router.
- Can be represented as a matrix:
 - Rows = Nodes
 - Columns = Nodes
 - Cell Value = Link Cost

Figure 4.63 *Example of a link-state database*



a. The weighted graph

	A	B	C	D	E	F	G
A	0	2	∞	3	∞	∞	∞
B	2	0	5	∞	4	∞	∞
C	∞	5	0	∞	∞	4	3
D	3	∞	∞	0	5	∞	∞
E	∞	4	∞	5	0	2	∞
F	∞	∞	4	∞	2	0	1
G	∞	∞	3	∞	∞	1	0

b. Link state database

Flooding and Link-State Packets (LSP)

- Routers discover neighbors using greeting messages.
- Each router creates a Link-State Packet (LSP).

LSP contains:

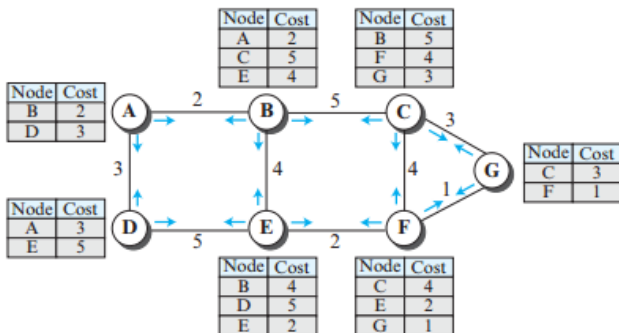
- Neighbor identity
- Link cost



Flooding Process:

- Router sends LSP through all interfaces.
- Newer LSP replaces older copy.
- LSP is forwarded through all interfaces except the arrival interface.

Figure 4.64 *LSPs created and sent out by each node to build LSDB*



Distance Vector vs Link State

Distance Vector Routing

Router tells neighbors what it knows about the entire network.

Link State Routing

Router tells the entire network what it knows about its neighbors.

- DV exchanges routing knowledge.
- LS exchanges link information.
- LS converges faster and avoids count-to-infinity problems.

Formation of Least-Cost Trees

- Every router uses the shared LSDB.
- Least-cost tree is created using:

Dijkstra's Algorithm

Steps:

- 1 Choose itself as root.
- 2 Add nearest node not yet in tree.
- 3 Update path costs.
- 4 Repeat until all nodes are included.

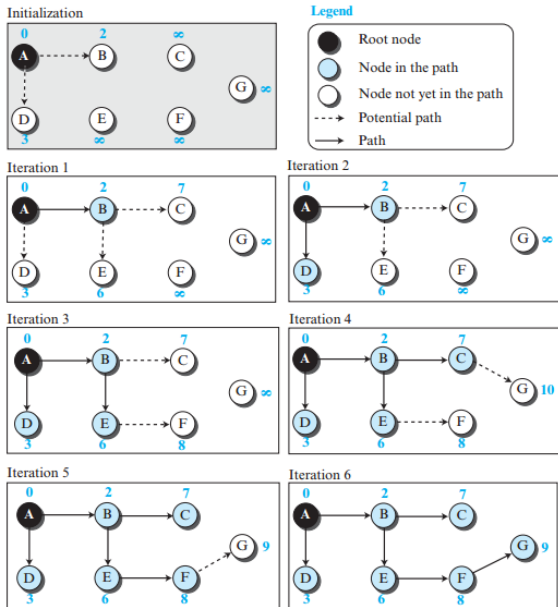
Dijkstra Algorithm

- 1 Initialize tree with source node.
- 2 Assign costs using LSDB.
- 3 Select closest node outside tree.
- 4 Add node to tree.
- 5 Recalculate path costs.
- 6 Continue until all nodes are included.

Table 4.5 *Dijkstra's Algorithm*

1	Dijkstra's Algorithm ()
2	{
3	<i>// Initialization</i>
4	Tree = {root} <i>// Tree is made only of the root</i>
5	for (y = 1 to N) <i>// N is the number of nodes</i>
6	{
7	if (y is the root)
8	D[y] = 0 <i>// D[y] is shortest distance from root to node y</i>
9	else if (y is a neighbor)
10	D[y] = c[root][y] <i>// c[x][y] is cost between nodes x and y in LSDB</i>
11	else
12	D[y] = ∞
13	}
14	<i>// Calculation</i>
15	repeat
16	{
17	find a node w, with D[w] minimum among all nodes not in the Tree
18	Tree = Tree $\cup$ {w} <i>// Add w to tree</i>
19	<i>// Update distances for all neighbor of w</i>
20	for (every node x, which is neighbor of w and not in the Tree)
21	{
22	D[x] = min{D[x], (D[w] + c[w][x])}
23	}
24	} until (all nodes included in the Tree)
25	} <i>// End of Dijkstra</i>

Figure 4.65 Least-cost tree



Path-Vector Routing (PV)

- Not based on least-cost routing.
- Source determines the best route.
- Routing decisions are policy-based.

Possible policies:

- Minimum number of hops.
- Avoid specific routers.
- Security requirements.
- Administrative preferences.

Goal

Allow the source to control the route taken by packets.

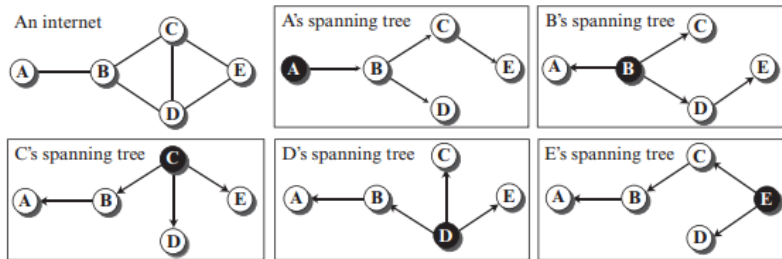
Spanning Trees in Path-Vector Routing

- Uses best spanning tree instead of least-cost tree.
- Tree depends on source policies.
- Different sources may create different spanning trees.

Examples:

- Avoid unsafe routers.
- Avoid competitor networks.
- Minimize number of intermediate nodes.

Figure 4.66 *Spanning trees in path-vector routing*



Creating Path Vectors

- Each node discovers immediate neighbors.
- Creates an initial path vector.
- Sends vector to neighboring nodes.
- Updates vector when new information arrives.

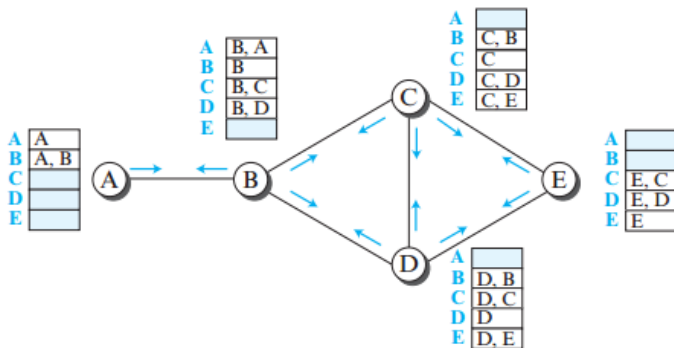
Path Vector Update Equation:

$$Path(x, y) = Best\{Path(x, y), [x + Path(v, y)]\}$$

where:

- v = Neighbor node
- Best path chosen according to policy

Figure 4.67 Path vectors made at booting time



Loop Prevention in Path-Vector Routing

- A path containing the current node is discarded.
- Prevents routing loops.

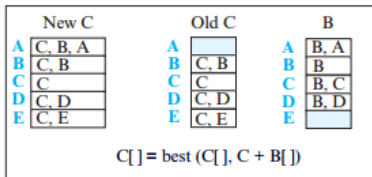
Rule:

If $Path(v, y)$ contains x , discard it

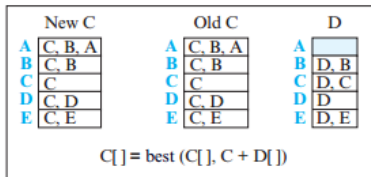
Benefits:

- Loop-free routing.
- Policy-based path selection.
- Better administrative control.

Figure 4.68 *Updating path vectors*



Event 1: C receives a copy of B's vector



Event 2: C receives a copy of D's vector

Note:

X []: vector X

Y: node Y

Path-Vector Algorithm

Initialization:

- Discover neighbors.
- Create initial path vector.
- Send vector to neighbors.

Update Process:

- Receive neighbor vector.
- Apply routing policy.
- Update path vector.
- Forward updated vector.

Table 4.6 Path-vector algorithm for a node

```

1  Path_Vector_Routing ( )
2  {
3      // Initialization
4      for (y = 1 to N)
5      {
6          if (y is myself)
7              Path[y] = myself
8          else if (y is a neighbor)
9              Path[y] = myself + neighbor node
10         else
11             Path[y] = empty
12     }
13     Send vector {Path[1], Path[2], ..., Path[y]} to all neighbors
14     // Update
15     repeat (forever)
16     {
17         wait (for a vector Pathw from a neighbor w)
18         for (y = 1 to N)
19         {
20             if (Pathw includes myself)
21                 discard the path // Avoid any loop
22             else
23                 Path[y] = best {Path[y], (myself + Pathw[y])}
24         }
25         If (there is a change in the vector)
26             Send vector {Path[1], Path[2], ..., Path[y]} to all neighbors
27     }
28 } // End of Path Vector

```

Routing Algorithms Comparison

Feature	Distance Vector	Link State / Path Vector
Information Shared	Routing Table	Link States / Paths
Algorithm	Bellman-Ford	Dijkstra / Policy-Based
Knowledge of Network	Partial	Complete (LS)
Loop Problems	Possible	Less Likely
Routing Goal	Least Cost	Least Cost / Policy

$DV \rightarrow \text{Bellman-Ford}$ $LS \rightarrow \text{Dijkstra}$ $PV \rightarrow \text{Policy Based}$

Unicast Routing Protocols

- Routing algorithms define how routes are calculated.
- Routing protocols implement these algorithms in the Internet.
- A protocol defines:
 - Domain of operation
 - Message exchange
 - Router communication
 - Interaction with other protocols

Major Internet Routing Protocols

- RIP (Distance-Vector Based)
- OSPF (Link-State Based)
- BGP (Path-Vector Based)

Internet Structure

- Modern Internet uses a multi-backbone architecture.
- Several private communication companies operate global backbones.
- Backbones are connected through peering points.

Hierarchy:

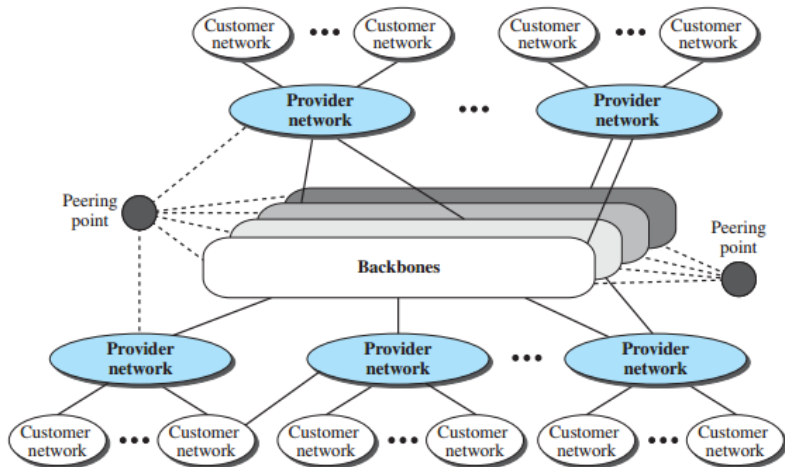
- 1 Global Backbones
- 2 Provider Networks
- 3 Consumer Networks



Note

Any of these entities can be considered an Internet Service Provider (ISP).

Figure 4.69 *Internet structure*



Need for Hierarchical Routing

Why not use one routing protocol for the entire Internet?

1. Scalability Problem

- Huge forwarding tables
- Slow route lookup
- Excessive update traffic

2. Administrative Issue

- Different organizations manage different networks.
- Organizations may use:
 - Different router vendors
 - Different routing algorithms
 - Different routing policies

Hierarchical Routing in the Internet

- Internet is divided into Autonomous Systems (AS).
- Each AS manages its own internal routing.
- Global routing connects all ASs together.

Routing Categories

- Intra-AS Routing
 - Intradomain Routing
 - Interior Gateway Protocol (IGP)
- Inter-AS Routing
 - Interdomain Routing
 - Exterior Gateway Protocol (EGP)



Internet Routing

- **Intradomain (IGP)**

- RIP
- OSPF

- **Interdomain (EGP)**

- BGP



Current Practice

RIP and OSPF are commonly used within an AS, while BGP is used between ASs.

Autonomous Systems (AS)

- An Autonomous System (AS) is a collection of networks and routers under one administrative authority.
- Each AS receives a unique Autonomous System Number (ASN).



ASN = 16-bit Unsigned Integer

- Assigned by ICANN.
- Uniquely identifies an AS.

Types of Autonomous Systems

1. Stub AS

- Connected to only one AS.
- Traffic originates or terminates here.
- Does not forward transit traffic.

2. Multihomed AS

- Connected to multiple ASs.
- Does not allow transit traffic.

3. Transient AS

- Connected to multiple ASs.
- Allows traffic to pass through.

Characteristics of AS Types

AS Type	Connections	Transit Traffic
Stub AS	One AS	Not Allowed
Multihomed AS	Multiple ASs	Not Allowed
Transient AS	Multiple ASs	Allowed

Examples:

- Stub AS → Customer Network
- Multihomed AS → Multi-provider Customer Network
- Transient AS → Provider Networks, Backbones

Unicast Routing Protocols: Summary

- Internet routing is hierarchical.
- Internet is divided into Autonomous Systems (AS).
- Each AS controls its own routing policies.
- Intra-AS routing uses:
 - RIP
 - OSPF
- Inter-AS routing uses:
 - BGP



$RIP \rightarrow DV$	$OSPF \rightarrow LS$	$BGP \rightarrow PV$
----------------------	-----------------------	----------------------

Routing Information Protocol (RIP)

- RIP is a widely used Intradomain Routing Protocol (IGP).
- Based on the **Distance-Vector Routing Algorithm**.
- Originally developed from Xerox Network System (XNS).
- Popularized through BSD UNIX.

Key Features

- Uses **hop count** as routing metric.
- Advertises routes to neighboring routers.
- Suitable for small to medium-sized Autonomous Systems.

Hop Count in RIP

- Cost is measured as the number of hops.
- A hop represents a network/router traversal.
- Smaller hop count = Better route.

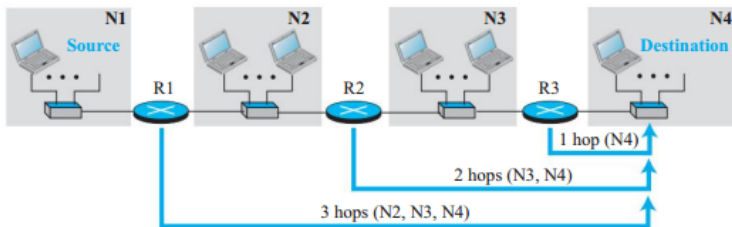
Important Limits

- Maximum valid hop count = 15
- Hop count 16 = Infinity (Unreachable)

Limitation

RIP can be used only in networks whose diameter is less than 15 hops.

Figure 4.70 *Hop counts in RIP*



RIP Forwarding Table

Each RIP router maintains a forwarding table containing:

Destination Network	Next Router	Hop Count
---------------------	-------------	-----------

Purpose

- Destination Network → Where packet must go.
- Next Router → Next hop router.
- Hop Count → Cost of the route.

Figure 4.71 *Forwarding tables*

Forwarding table for R1			Forwarding table for R2			Forwarding table for R3		
Destination network	Next router	Cost in hops	Destination network	Next router	Cost in hops	Destination network	Next router	Cost in hops
N1	—	1	N1	R1	2	N1	R2	3
N2	—	1	N2	—	1	N2	R2	2
N3	R2	2	N3	—	1	N3	—	1
N4	R2	3	N4	R3	2	N4	—	1

RIP Operation and Messages

Implementation

- Runs as an application-layer protocol.
- Uses UDP port 520.
- Common UNIX daemon: `routed`

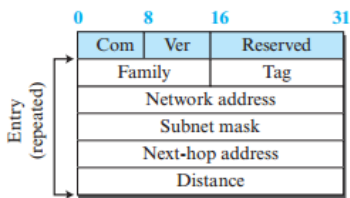
RIP Message Types

- **Request** – Ask for routing information.
- **Response (Update)** – Send routing information.

Updates are sent

- Periodically (every 30 seconds)
- Whenever routing information changes

Figure 4.72 *RIP message format*



Fields

Com: Command, request (1), response (2)

Ver: Version, current version is 2

Family: Family of protocol, for TCP/IP value is 2

Tag: Information about autonomous system

Network address: Destination address

Subnet mask: Prefix length

Next-hop address: Address length

Distance: Number of hops to the destination

Timers and Performance of RIP

RIP Timers

- Periodic Timer: 25–35 s update interval
- Expiration Timer: 180 s
- Garbage Collection Timer: 120 s

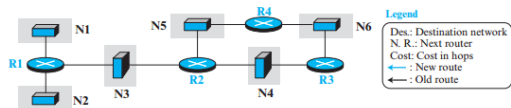
Advantages

- Simple and easy to implement
- Low routing overhead

Limitations

- Maximum 15 hops
- Slow convergence
- Susceptible to routing loops
- Count-to-Infinity problem

Figure 4.73 Example of an autonomous system using RIP



R1			R2			R3			R4		
Des.	N. R.	Cost	Des.	N. R.	Cost	Des.	N. R.	Cost	Des.	N. R.	Cost
N1	—	1	N3	—	1	N4	—	1	N5	—	1
N2	—	1	N4	—	1	N6	—	1	N6	—	1
N3	—	1	N5	—	1						

Forwarding tables after all routers booted

New R1			Old R1			R2 Seen by R1		
Des.	N. R.	Cost	Des.	N. R.	Cost	Des.	N. R.	Cost
N1	—	1	N1	—	1	N3	R2	2
N2	—	1	N2	—	1	N4	R2	2
N3	—	1	N3	—	1	N5	R2	2
N4	R2	2						
N5	R2	2						

New R3			Old R3			R2 Seen by R3		
Des.	N. R.	Cost	Des.	N. R.	Cost	Des.	N. R.	Cost
N3	R2	2	N4	—	1	N3	R2	2
N4	—	1	N6	—	1	N4	R2	2
N5	R2	2				N5	R2	2
N6	—	1						

New R4			Old R4			R2 Seen by R4		
Des.	N. R.	Cost	Des.	N. R.	Cost	Des.	N. R.	Cost
N3	R2	2	N5	—	1	N3	R2	2
N4	R2	2	N6	—	1	N4	R2	2
N5	—	1				N5	R2	2
N6	—	1						

Changes in the forwarding tables of R1, R3, and R4 after they receive a copy of R2's table

Final R1			Final R2			Final R3			Final R4		
Des.	N. R.	Cost	Des.	N. R.	Cost	Des.	N. R.	Cost	Des.	N. R.	Cost
N1	—	1	N1	R1	2	N1	R2	3	N1	R2	3
N2	—	1	N2	R1	2	N2	R2	3	N2	R2	3
N3	—	1	N3	—	1	N3	R2	2	N3	R2	2
N4	R2	2	N4	—	1	N4	—	1	N4	R2	2
N5	R2	2	N5	—	1	N5	R2	2	N5	—	1
N6	R2	3	N6	R3	2	N6	—	1	N6	—	1

Forwarding tables for all routers after they have been stabilized

Open Shortest Path First (OSPF)

- OSPF is an Intradomain Routing Protocol (IGP).
- Based on the **Link-State Routing Algorithm**.
- Open standard protocol developed by IETF.
- Designed for both small and large Autonomous Systems (AS).

Key Features

- Uses Dijkstra's Shortest Path Algorithm.
- Faster convergence than RIP.
- Supports hierarchical routing using areas.

OSPF Metric and Forwarding

- OSPF assigns a cost to each link.
- Cost may depend on:
 - Bandwidth
 - Delay
 - Reliability
 - Hop Count
- Lowest total cost path is selected.

Forwarding Table

- Created from the shortest-path tree.
- Generated using Dijkstra's Algorithm.



Figure 4.74 *Metric in OSPF*

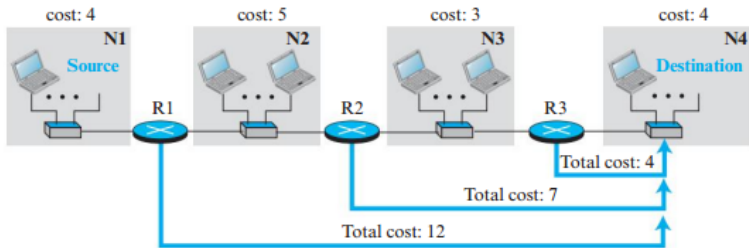


Figure 4.75 Forwarding tables in OSPF

Forwarding table for R1			Forwarding table for R2			Forwarding table for R3		
Destination network	Next router	Cost	Destination network	Next router	Cost	Destination network	Next router	Cost
N1	—	4	N1	R1	9	N1	R2	12
N2	—	5	N2	—	5	N2	R2	8
N3	R2	8	N3	—	3	N3	—	3
N4	R2	12	N4	R3	7	N4	—	4

OSPF Areas and Link-State Advertisements

Areas

- Large AS is divided into smaller areas.
- Reduces flooding traffic.
- Backbone Area (Area 0) connects all areas.

Link-State Advertisements (LSAs)

- Router Link
- Network Link
- Summary Link to Network
- Summary Link to AS Border Router
- External Link

Figure 4.76 *Areas in an autonomous system*

Autonomous System (AS)

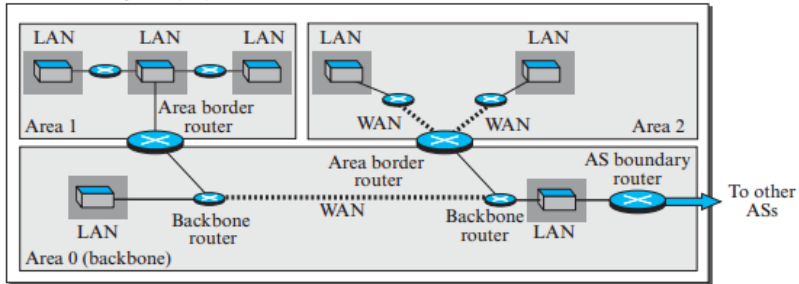


Figure 4.77 Five different LSPs

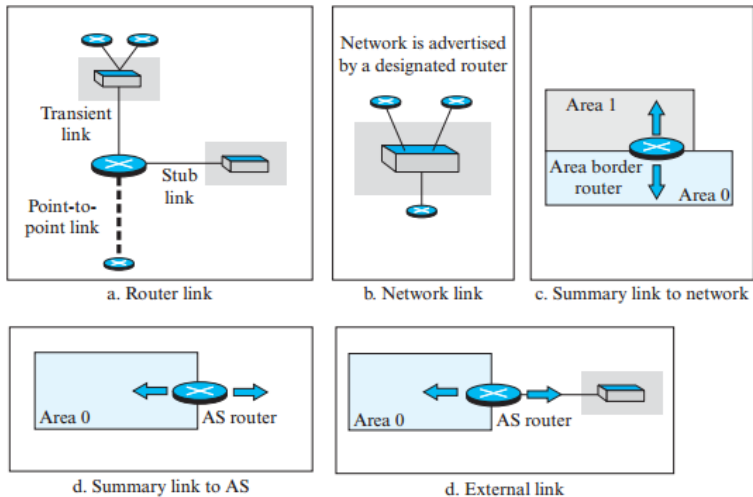
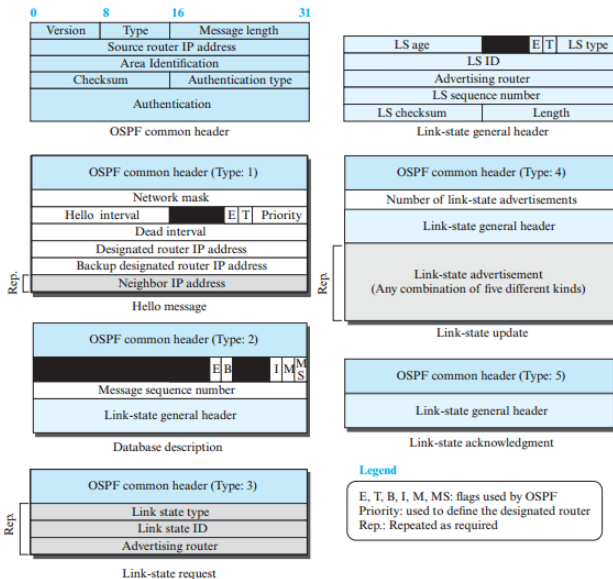


Figure 4.78 OSPF message formats



OSPF Messages and Performance

OSPF Message Types

- Hello
- Database Description
- Link-State Request
- Link-State Update
- Link-State Acknowledgment

Advantages

- Fast convergence
- Supports large networks
- More robust than RIP

Limitation

- More complex than RIP
- Flooding may create overhead

Border Gateway Protocol (BGP)

- BGP (Border Gateway Protocol) is the routing protocol used between Autonomous Systems (ASs).
- Current version: BGP-4.
- Based on the Path-Vector Routing Algorithm.
- Provides reachability information between networks.
- Used by Internet Service Providers (ISPs) to exchange routing information.

Key Point

BGP is the only Interdomain Routing Protocol used in the Internet today.

Need for BGP

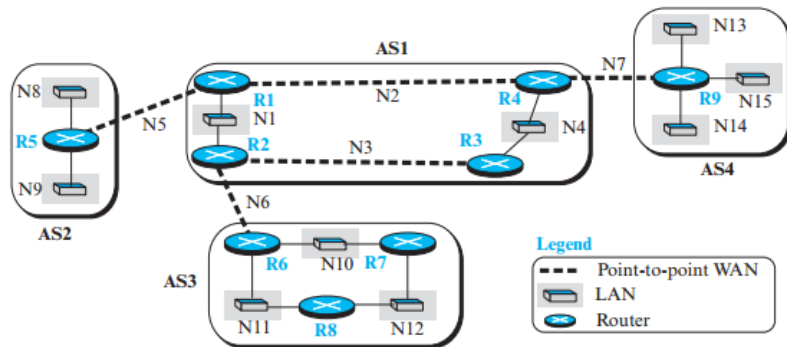
- Routers inside an AS use RIP or OSPF.
- These protocols cannot exchange routing information between different ASs.
- BGP enables communication among Autonomous Systems.

AS1 ↔ AS2 ↔ AS3 ↔ AS4

Example

- A packet from AS2 to AS4 must travel through AS1.
- BGP provides this interdomain routing information.

Figure 4.79 A sample internet with four ASs



Types of BGP

External BGP (eBGP)

- Runs between routers of different ASs.
- Exchanges routing information across AS boundaries.
- Establishes sessions between border routers.

Internal BGP (iBGP)

- Runs between routers within the same AS.
- Distributes BGP information inside the AS.
- Uses a full-mesh connectivity model.

Insert Figure 4.80 and Figure 4.81

Figure 4.80 *eBGP operation*

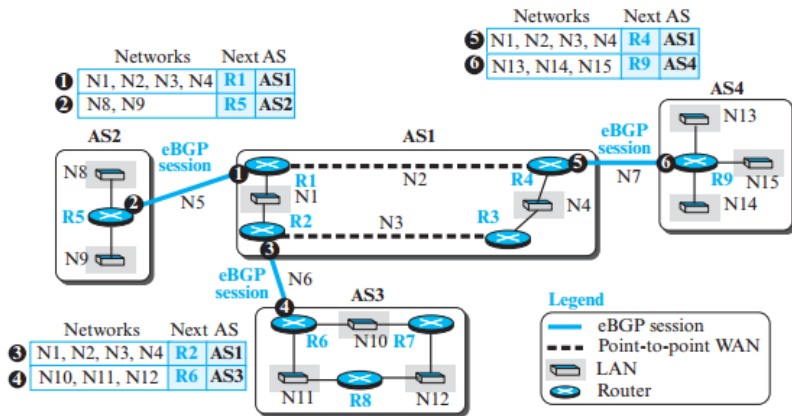
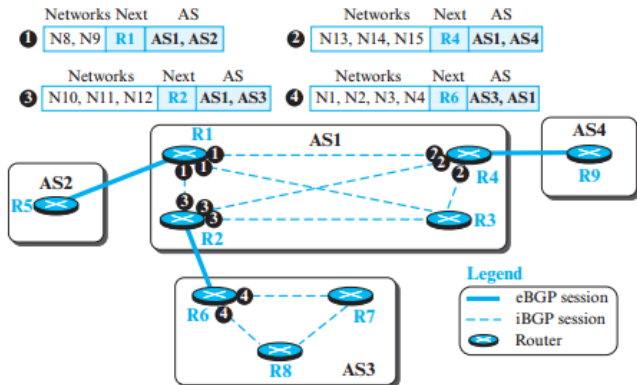


Figure 4.81 *Combination of eBGP and iBGP sessions in our internet*



BGP Peers and Sessions

- Routers running BGP are called **BGP Speakers** or **Peers**.
- BGP communication uses TCP connections.
- TCP Port Number: 179.
- Each TCP connection is called a **BGP Session**.

Example

Router R1 and Router R5 establish an eBGP session and exchange routing updates.

- eBGP sessions connect different ASs.
- iBGP sessions distribute routing information inside an AS.

Figure 4.82 *Finalized BGP path tables*

Networks	Next	Path	Networks	Next	Path	Networks	Next	Path
N8, N9	R5	AS1, AS2	N8, N9	R1	AS1, AS2	N8, N9	R2	AS1, AS2
N10, N11, N12	R2	AS1, AS3	N10, N11, N12	R6	AS1, AS3	N10, N11, N12	R2	AS1, AS3
N13, N14, N15	R4	AS1, AS4	N13, N14, N15	R1	AS1, AS4	N13, N14, N15	R4	AS1, AS4
Path table for R1			Path table for R2			Path table for R3		
Networks	Next	Path	Networks	Next	Path	Networks	Next	Path
N8, N9	R1	AS1, AS2	N1, N2, N3, N4	R1	AS2, AS1	N1, N2, N3, N4	R2	AS3, AS1
N10, N11, N12	R1	AS1, AS3	N10, N11, N12	R1	AS2, AS1, AS3	N8, N9	R2	AS3, AS1, AS2
N13, N14, N15	R9	AS1, AS4	N13, N14, N15	R1	AS2, AS1, AS4	N13, N14, N15	R2	AS3, AS1, AS4
Path table for R4			Path table for R5			Path table for R6		
Networks	Next	Path	Networks	Next	Path	Networks	Next	Path
N1, N2, N3, N4	R6	AS3, AS1	N1, N2, N3, N4	R6	AS3, AS1	N1, N2, N3, N4	R4	AS4, AS1
N8, N9	R6	AS3, AS1, AS2	N8, N9	R6	AS3, AS1, AS2	N8, N9	R4	AS4, AS1, AS2
N13, N14, N15	R6	AS3, AS1, AS4	N13, N14, N15	R6	AS3, AS1, AS4	N10, N11, N12	R4	AS4, AS1, AS3
Path table for R7			Path table for R8			Path table for R9		

Figure 4.83 Forwarding tables after injection from BGP

Des.	Next	Cost
N1	—	1
N4	R4	2
N8	R5	1
N9	R5	1
N10	R2	2
N11	R2	2
N12	R2	2
N13	R4	2
N14	R4	2
N15	R4	2

Table for R1

Des.	Next	Cost
N1	—	1
N4	R3	2
N8	R1	2
N9	R1	2
N10	R6	1
N11	R6	1
N12	R6	1
N13	R3	3
N14	R3	3
N15	R3	3

Table for R2

Des.	Next	Cost
N1	R2	2
N4	—	1
N8	R2	3
N9	R2	3
N10	R2	2
N11	R2	2
N12	R2	2
N13	R4	2
N14	R4	2
N15	R4	2

Table for R3

Des.	Next	Cost
N1	R1	2
N4	—	1
N8	R1	2
N9	R1	2
N10	R3	3
N11	R3	3
N12	R3	3
N13	R9	1
N14	R9	1
N15	R9	1

Table for R4

Des.	Next	Cost
N8	—	1
N9	—	1
0	R1	1

Table for R5

Des.	Next	Cost
N10	—	1
N11	—	1
N12	R7	2
0	R2	1

Table for R6

Des.	Next	Cost
N10	—	1
N11	R6	2
N12	—	1
0	R6	2

Table for R7

Des.	Next	Cost
N10	R6	2
N11	—	1
N12	—	1
0	R6	2

Table for R8

Des.	Next	Cost
N13	—	1
N14	—	1
N15	—	1
0	R4	1

Table for R9

BGP Reachability Information

- BGP exchanges information about reachable networks.
- Border routers advertise networks available in their AS.
- Routers build path tables based on received updates.
- These path tables are later injected into RIP or OSPF forwarding tables.

Goal

Enable every router in the Internet to determine how to reach remote networks.

Path Attributes in BGP

A BGP route contains additional information called **Path Attributes**.

- ORIGIN
- AS-PATH
- NEXT-HOP
- MULTI EXIT DISC (MED)
- LOCAL-PREF
- ATOMIC AGGREGATE
- AGGREGATOR

Purpose

- Route selection
- Loop prevention
- Policy-based routing

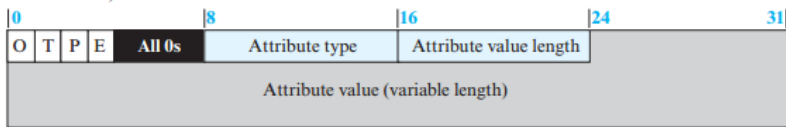
Figure 4.84 *Format of path attribute*

O: Optional bit (set if attribute is optional)

P: Partial bit (set if an optional attribute is lost in transit)

T: Transitive bit (set if attribute is transitive)

E: Extended bit (set if attribute length is two bytes)



Important BGP Attributes

ORIGIN

- Indicates source of routing information.

AS-PATH

- Lists Autonomous Systems along the route.
- Prevents routing loops.

NEXT-HOP

- Specifies the next router for forwarding.

LOCAL-PREF

- Indicates preferred route within an AS.

Route Selection in BGP

When multiple routes exist, BGP selects the best route.

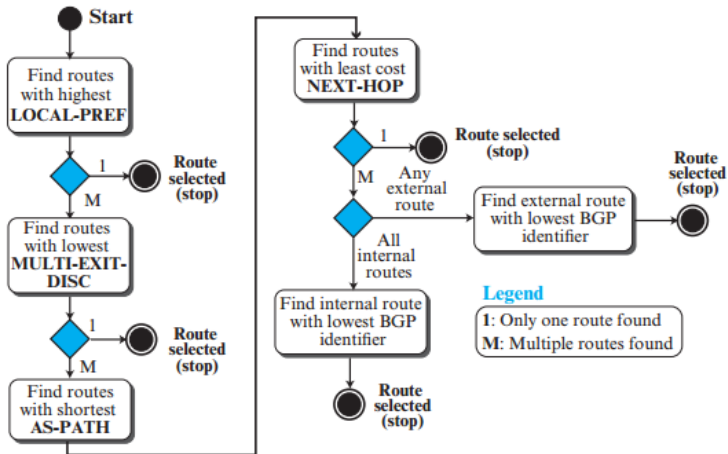
Typical Selection Criteria

- 1 Highest Local Preference
- 2 Shortest AS-PATH
- 3 Lowest MED
- 4 Prefer eBGP over iBGP
- 5 Lowest Router ID

Goal

Choose the most preferred and efficient route according to routing policies.

Figure 4.85 Flow diagram for route selection



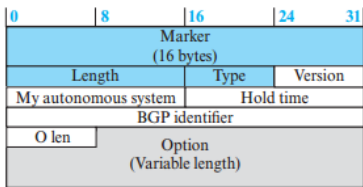
BGP uses four message types:

- 1 Open Message
- 2 Update Message
- 3 Keepalive Message
- 4 Notification Message

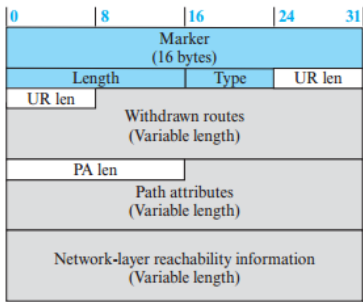
Functions

- Open – Establish BGP session.
- Update – Advertise or withdraw routes.
- Keepalive – Maintain connection.
- Notification – Report errors and terminate session.

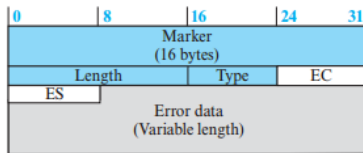
Figure 4.86 BGP messages



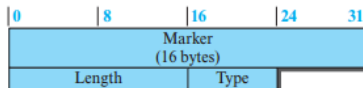
Open message (type 1)



Update message (type 2)



Notification message (type 3)



Keepalive message (type 4)

Fields in common header

Marker: Reserved for authentication

Length: Length of total message in bytes

Type: Type of message (1 to 4)

Abbreviations

O len: Option length

EC: Error code

ES: Error subcode

UR len: Unfeasible route length

PA len: Path attribute length

Address Aggregation in BGP

- BGP supports address aggregation (route summarization).
- Multiple destination networks can be represented by a single prefix.
- Reduces routing table size.
- Improves scalability of Internet routing.

Example

- 14.18.20.0/26
- 14.18.20.64/26
- 14.18.20.128/26
- 14.18.20.192/26

Can be aggregated as:

14.18.20.0/24



Advantages and Limitations of BGP

Advantages

- Internet-scale routing.
- Policy-based route selection.
- Loop prevention using AS-PATH.
- Supports route aggregation.
- Highly scalable.

Limitations

- Complex configuration.
- Slow convergence.
- Large routing tables.

Comparison of Routing Protocols

Feature	RIP	OSPF	BGP
Routing Type	Distance Vector	Link State	Path Vector
Scope	Intra-AS	Intra-AS	Inter-AS
Algorithm	Bellman-Ford	Dijkstra	Path Vector
Metric	Hop Count	Cost	Path Attributes
Protocol	UDP	IP (89)	TCP (179)
Scalability	Low	High	Very High
Convergence	Slow	Fast	Moderate

Summary

- RIP → Small Networks
- OSPF → Enterprise Networks
- BGP → Internet Routing